

Semana 1 Introducción y Búsqueda Desinformada

ISIS-1611: Inteligencia Artificial

Carla González

9 de junio de 2026

1. Contexto e Historia de la IA

1.1. Impacto de la IA en el mundo real

La IA impacta múltiples sectores: economía, política, ley y derecho, trabajo, ciencia, salud y educación.

1.2. ¿Qué es la Inteligencia Artificial?

Existen cuatro enfoques para definir la IA:

Pensamiento humano Modelos cognitivos	Pensamiento racional Leyes del pensamiento (lógica)
Actuar humanamente Test de Turing	Actuar racionalmente Agentes racionales

Concepto clave

La IA moderna se enfoca en **agentes racionales**: entidades que perciben su entorno y actúan para maximizar su medida de desempeño esperada.

1.3. Historia de la IA

2. Agentes Inteligentes

2.1. ¿Qué es un Agente?

Definición 2.1 (Agente). Un agente es cualquier entidad que **percibe** su ambiente mediante **sensores** y **actúa** sobre él mediante **actuadores**.

La función del agente mapea percepciones a acciones: $f : P^* \rightarrow A$

2.2. Racionalidad

Un agente racional selecciona la acción que maximiza su **medida de desempeño** esperada, dado su conocimiento actual.

Depende de:

1. La medida de desempeño
2. El conocimiento previo del entorno
3. Las acciones disponibles
4. La secuencia de percepciones hasta el momento

2.3. Descripción PEAS

Para caracterizar el ambiente de una tarea se usa PEAS:

- **Performance** (Medida de desempeño)
- **Environment** (Entorno)
- **Actuators** (Actuadores)
- **Sensors** (Sensores)

2.4. Naturaleza de los Ambientes

Dimensión	Valores posibles
Observabilidad	Completamente observable / Parcialmente observable
Agentes	Un solo agente / Multiagente
Determinismo	Determinista / Estocástico
Episodicidad	Episódico / Secuencial
Dinamismo	Estático / Dinámico
Tiempo	Discreto / Continuo
Conocimiento	Conocido / Desconocido

2.5. Tipos de Agentes

1. **Agente de reflejo simple:** actúa según la percepción actual (reglas condición-acción).
2. **Agente de reflejo basado en modelos:** mantiene un estado interno del mundo.
3. **Agente basado en objetivos:** considera metas para elegir acciones.
4. **Agente basado en utilidad:** maximiza una función de utilidad.
5. **Agente que aprende:** mejora su desempeño con la experiencia.

3. Búsqueda Desinformada I: Formulación del Problema

3.1. Formulación de un Problema de Búsqueda

Definición 3.1 (Problema de búsqueda). Un problema de búsqueda se define con cinco componentes:

1. **Estado inicial:** s_0
2. **Modelo de transición:** $\text{RESULT}(s, a) \rightarrow s'$
3. **Función de costo de acción:** $c(s, a, s')$
4. **Test de meta:** $\text{IS-GOAL}(s) \rightarrow \{T, F\}$
5. **Espacio de estados:** grafo de todos los estados alcanzables

La **solución** es una secuencia de acciones que lleva del estado inicial a un estado meta. Una **solución óptima** minimiza el costo total del camino.

3.2. Nodos vs. Estados

Un **nodo** en el árbol de búsqueda contiene:

- El estado que representa
- El nodo padre
- La acción que generó el nodo
- El costo del camino $g(n)$
- La profundidad

3.3. Medidas de Desempeño de Algoritmos

Criterio	Descripción
Compleitud	¿Encuentra solución si existe?
Optimalidad	¿Encuentra la solución de menor costo?
Complejidad en tiempo	Nodos generados/expandidos
Complejidad en espacio	Nodos en memoria simultáneamente

Los parámetros usuales son b (factor de ramificación), d (profundidad de la solución más superficial) y m (profundidad máxima del árbol).

4. Búsqueda Desinformada II: Algoritmos

4.1. Breadth-First Search (BFS)

- Expande el nodo más **superficial** primero (FIFO).
- **Completo**: sí (si b finito).
- **Óptimo**: sí, si todos los costos de paso son iguales.
- Tiempo: $\mathcal{O}(b^d)$ Espacio: $\mathcal{O}(b^d)$

4.2. Depth-First Search (DFS)

- Expande el nodo más **profundo** primero (LIFO).
- **Completo**: no en árbol infinito; sí en grafo finito.
- **Óptimo**: no.
- Tiempo: $\mathcal{O}(b^m)$ Espacio: $\mathcal{O}(bm)$

4.3. Depth-Limited Search y Iterative Deepening (IDDFS)

IDDFS ejecuta DFS con límite de profundidad creciente $l = 0, 1, 2, \dots$

- Combina las ventajas de BFS (completo, óptimo si costos iguales) con las de DFS (espacio lineal).
- Tiempo: $\mathcal{O}(b^d)$ Espacio: $\mathcal{O}(bd)$

4.4. Uniform-Cost Search (UCS)

- Expande el nodo con menor **costo de camino** $g(n)$ (cola de prioridad).
- Equivale a BFS cuando todos los costos son iguales.
- **Completo**: sí (si costos $> \epsilon > 0$).
- **Óptimo**: sí.
- Tiempo y espacio: $\mathcal{O}(b^{1+\lceil C^*/\epsilon \rceil})$, donde C^* es el costo óptimo.

4.5. Comparativa Búsqueda Desinformada

Algoritmo	Completo	Óptimo	Tiempo	Espacio
BFS	Sí	Sí*	b^d	b^d
DFS	No	No	b^m	bm
IDDFS	Sí	Sí*	b^d	bd
UCS	Sí	Sí	$b^{C^*/\epsilon}$	$b^{C^*/\epsilon}$

*Óptimo si costos de paso son iguales

5. Búsqueda Informada: Heurísticas

5.1. Función Heurística

Definición 5.1 (Heurística). $h(n)$ es una función que estima el costo desde el nodo n hasta la meta más cercana. Debe ser **admisible**: nunca sobreestima el costo real.

Importante

Una heurística es **admisible** si $h(n) \leq h^*(n)$ para todo n , donde $h^*(n)$ es el costo real al llegar a la meta.

5.2. Greedy Best-First Search

Expande el nodo que parece más cercano a la meta según $h(n)$.

- Función de evaluación: $f(n) = h(n)$
- **No es óptimo** ni completo en general.
- Rápido en la práctica, puede quedar atrapado en mínimos locales.

5.3. A*

Combina el costo real del camino y la heurística:

$$f(n) = g(n) + h(n)$$

- $g(n)$: costo del camino desde inicio hasta n
- $h(n)$: estimación del costo desde n hasta la meta
- **Completo** y **óptimo** si h es admisible (búsqueda en árbol) o consistente (búsqueda en grafo).

Definición 5.2 (Consistencia / Monotonía). h es consistente si para todo nodo n y sucesor n' :

$$h(n) \leq c(n, a, n') + h(n')$$

La consistencia implica admisibilidad.

6. Repaso Quiz

6.1. Preguntas de verdadero o falso

1. Un agente que percibe solo información parcial acerca del estado no puede ser perfectamente racional: **Falso** la racionalidad depende de la expectativa de rendimiento dada la información que el agente posee (el historial de percepciones), no del éxito omnisciente o del estado real completo del mundo. Un agente en un entorno parcialmente observable puede tomar la decisión óptima posible con base en sus percepciones parciales y seguir siendo perfectamente racional.
2. Existen ambientes de tareas en los que ningún agente de reflejo puro se puede comportar de forma racional: **Verdadero** en entornos parcialmente observables, las acciones óptimas suelen depender de la historia o de estados ocultos que no se pueden deducir únicamente de la percepción actual; un agente de reflejo puro carece de memoria y puede entrar en bucles infinitos.
3. Es posible para un agente dado ser perfectamente racional en dos ambientes de tarea diferentes: **Verdadero**, si las acciones requeridas para maximizar la medida de rendimiento en ambos entornos coinciden ante las mismas percepciones, el agente será racional en ambos.
4. Todos los agentes son racionales en un ambiente no observable: **Falso**, que el entorno sea no observable no significa que cualquier acción sea buena. El agente debe actuar basándose en estimaciones o en su conocimiento inicial (suponer estados). Un agente que tome decisiones autodestructivas a ciegas seguirá siendo irracional, mientras que uno racional calculará la secuencia de acciones que estadísticamente tenga mayor probabilidad de éxito.
5. Un agente de poker perfectamente racional nunca pierde: **Falso**, el poker es un entorno estocástico y de información oculta, un agente perfectamente racional tomará la decisión con *el mayor valor esperado*, pero la suerte puede hacer que pierda manos o partidas.

6.2. Descripción PEAS y caracterización del ambiente: Rutina de piso de gimnasia

P (Rendimiento/performance): Puntuación de los jueces (dificultad, ejecución, fluidez, sincronización, técnica, no salirse del tapete) **E (Entorno/Environment):** El tapete de gimnasia, la gravedad, las capacidades físicas del cuerpo del gimnasta. **A (Actuadores/Actuators):** Músculos (piernas, brazos, torso) para realizar giros, saltos, mortales y transiciones. **S (Sensores/Sensors):** Sistema vestibular (equilibrio), propiocepción, visión.

6.3. Caracterización del ambiente: Rutina de piso de gimnasia

Continuo o discreto? **Continuo** ya que involucra las posiciones corporales, fuerzas y tiempos que varían continuamente.

Total o parcialmente observable? **Totalmente observable** ya que el agente conoce su propio estado físico y la posición del piso.

Determinista o estocástico? **Estocástico** ya que a nivel físico real, pequeñas variaciones o fatiga muscular pueden alterar el resultado de un salto.

Episódico o Secuencial? **Secuencial** un mal salto inicial arruina el equilibrio para el siguiente movimiento.

Estatico o dinámico? **Estático** ya que el tapete no cambia de lugar por sí mismo mientras se ejecuta la rutina, aunque el tiempo corre.

Un agente o multiagente? **agente** ya que es una rutina individual.

6.4. Descripción PEAS y caracterización del ambiente: Jugar Futbol

P (Rendimiento/performance): Goles anotados, evitar goles en contra, ganar el partido, precisión de pases, posesión del balón. **E (Entorno/Environment):** Campo de juego, 11 rivales, 10 compañeros de equipo, condiciones climáticas. **A (Actuadores/Actuators):** Movimiento de las piernas (correr, patear, cambiar de dirección), torso, cabeza. **S (Sensores/Sensors):** Visión, audición, tacto/propiocepción.

6.5. Caracterización del ambiente: Jugar futbol

Continuo o discreto? **Continuo** ya que involucra las posiciones corporales, fuerzas y tiempos que varían continuamente.

Total o parcialmente observable? **parcialmente observable** ya que no se puede ver lo que pasa a la espalda del jugador sin voltear.

Determinista o estocástico? **Estocástico** física del balón, rebotes impredecibles.

Episódico o Secuencial? **Secuencial**

Estatico o dinámico? **Dinámico** ya que los rivales y compañeros se mueven continuamente mientras el agente decide.

Un agente o multiagente? **multiagente** ya que es competitivo con los rivales y cooperativo con los compañeros.

7. Formulación de problemas de búsqueda

1. Por qué la formulación debe estar guiada por una formulación de la meta? En un entorno real, los detalles del mundo son infinitos. Si no se define primero un **objetivo** claro, el agente no sabrá que aspectos del mundo son relevantes y cuales debe ignorar (*abstracción*). la meta sirve como criterio para establecer que constituye un estado satisfactorio, lo que permite acotar las acciones posibles a considerar y estructurar la función de costo y la prueba de meta. Sin ella el espacio de estados sería inmanejable.

2. El problema de las 6 cajas de cristal con llaves y una banana

El **estado** es un entero $i \in \{1, 2, 3, 4, 5, 6\}$ que representa la caja mas alta que el agente ha logrado desbloquear hasta el momento; o equivalentemente el numero de la caja que puede abrir actualmente porque tiene su llave. Inicialmente se tiene la llave de la primera. El **estado inicial** $S_0 = 1$ (el agente tiene la llave de la caja 1) El **modelo de transición** si el agente ejecuta **Abrir caja i**:

- Si $1 \leq i \leq 5$, el agente obtiene la llave de la caja $i + 1$, por lo que el nuevo estado es $i + 1$.
- Si $i=6$, el agente abre la ultima caja y obtiene la banana

La **prueba de meta** El estado actual es igual a haber abierto la caja 6 y tener la banana? Estado=6

El **costo de camino** es que cada acción de abrir una caja tiene un costo uniforme de 1.

3. Cuadrícula $n \times n$ para pintar el piso: El **Estado**: una tupla $\langle (x, y), P \rangle$ donde: (x, y) son las coordenadas actuales del agente ($1 \leq x, y \leq n$) P es un conjunto de pares ordenados o una matriz booleana de tamaño $n \times n$ que registra cuales celdas ya han sido pintadas. Las celdas que contienen "pozos sin fondo" no forman parte de las celdas transitables ni requieren pintura. El **Estado inicial** $\langle (x_0, y_0), P_0 \rangle$, donde (x_0, y_0) es la celda de inicio y $P_0 = \emptyset$ Las **acciones** son **pintar** (válido si $(x, y) \notin P$), y **moverse** a (x', y') donde (x', y') es adyacente ortogonalmente a (x, y) y es una celda de piso sin pintar. El **modelo de transición** es

si *Accion: Pintar* el nuevo estado es $\langle (x, y), P \cup (x, y) \rangle$. si *Accion: Moverse* a (x', y') el nuevo estado es $\langle (x', y'), P_0 \rangle$ La **prueba meta** es que el conjunto P contiene *todas* las celdas transitables de la cuadrícula que correspondían a pisos sin pintar.

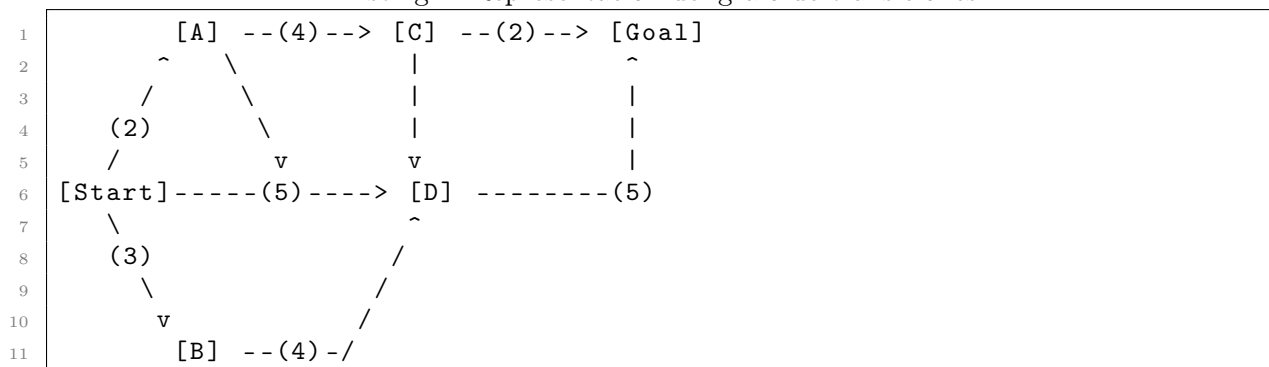
8. Búsqueda no informada: Algoritmos

1. DFS siempre expande al menos tantos nodos como la búsqueda A^* con una heurística admisible. **Falso**, DFS es un algoritmo propenso a la "suerte" del orden alfabético/de expansión. SI la meta se encuentra en la primera rama profunda que DFS decide explorar, puede encontrarla de manera inmediata expandiendo muchísimos menos nodos que los que A^* requeriría para verificar la optimalidad a lo ancho del árbol.
2. $h(n) = 0$ es una heurística admisible para el problema del 8-puzzle. **Verdadero**, una heurística es admisible si nunca sobreestima el costo real hacia la meta $h(n) \leq h^*(n)$. Dado que el costo real restante siempre es mayor igual a 0, definir $h(n) = 0$ para todo nodo es una subestimación válida y segura. al usar $h(n) = 0$, el algoritmo A^* se degrada exactamente al algoritmo de UCS.
3. A^* no es útil en robótica porque las percepciones, los estados y las acciones son continuos. **Falso**, aunque los espacios físicos reales son continuos, en robótica se aplican técnicas de *discretización*, por ejemplo la descomposición en celdas, mapas de ocupación o grafos de visibilidad. Una vez el espacio continuo se modela como un grafo discreto de posiciones válidas, A^* es uno de los algoritmos más utilizados para la planeación de trayectorias.
4. La búsqueda en amplitud BFS es completa incluso si se permiten costos de paso iguales a cero. **Verdadero** BFS expande de los nodos capa por capa según su **profundidad**, de manera completamente independiente de los costos numéricos acumulados $g(n)$. siempre que el factor de ramificación b sea finito, BFS explorará de manera exhaustiva todos los nodos a profundidad d antes de pasar a $d + 1$, garantizando encontrar la meta si esta existe.
5. La distancia Manhattan es una heurística admisible para el problema de mover una torre en ajedrez de A a B en el menor número de movimientos. **Falso**, la distancia Manhattan cuenta cuantas casillas ortogonales individuales separan dos puntos paso a paso. sin embargo, una torre de ajedrez puede saltar de un extremo al otro del tablero en **un solo movimiento** si la fila o columna está despejada. Por ende, el costo real puede ser de solo 1 movimiento, mientras que la distancia Manhattan daría un valor mucho mayor. Al sobreestimar el costo real, la heurística no es admisible.

8.1. Simulación de Estrategias de Búsqueda en Grafos

Reglas estipuladas: Los empates se resuelven **alfabéticamente**. En búsqueda de grafos se mantiene una lista de cerrados, expandiendo cada estado una sola vez.

Listing 1: Representación del grafo de transiciones



a. Búsqueda en Profundidad (DFS)

- **Frontera inicial (LIFO/Pila):** [Start]
- **Orden de expansión:** Start $\rightarrow$ A $\rightarrow$ C $\rightarrow$ D $\rightarrow$ Goal.
- **Camino devuelto:** Start $\rightarrow$ A $\rightarrow$ C $\rightarrow$ D $\rightarrow$ Goal.

b. Búsqueda en Amplitud (BFS)

- **Frontera inicial (FIFO/Cola):** [Start]
- **Orden de expansión:** Start, A, B, D.
- **Camino devuelto:** Start $\rightarrow$ D $\rightarrow$ Goal.

c. Búsqueda de Costo Uniforme (UCS)

- **Análisis de pasos en frontera de prioridad:**

1	Paso 1: Expandir Start (g=0). Frontera: [(A, g=2), (B, g=3), (D, g=5)]
2	Paso 2: Expandir A (g=2). Frontera: [(B, g=3), (D, g=5), (C, g=6)]
3	Paso 3: Expandir B (g=3). Frontera: [(D, g=5), (C, g=6)]
4	Paso 4: Expandir D (g=5). Frontera: [(C, g=6), (Goal, g=10)]
5	Paso 5: Expandir C (g=6). Frontera: [(Goal, g=8)]
6	Paso 6: Extraer Goal (g=8). Fin.

- **Orden de expansión:** Start, A, B, D, C, Goal.
- **Camino devuelto:** Start $\rightarrow$ A $\rightarrow$ C $\rightarrow$ Goal (Costo total óptimo: 8).

8.2. Código de BFS

```

1 def breadth_first_search(start, goal, get_neighbors):
2     frontera = deque()
3     frontera.append((start, [start]))
4     visitados = set()
5     visitados.add(start)
6     nodos_explorados = []
7     while frontera:
8         posi, camino = frontera.popleft()
9         nodos_explorados.append(posi)
10        if posi == goal:
11            return camino, nodos_explorados
12        for veci, costo in get_neighbors(posi):
13            if veci not in visitados:
14                visitados.add(veci)
15                if veci == goal:
16                    return camino + [veci],
17                        nodos_explorados
18                frontera.append((veci, camino + [veci]))
19    return [], nodos_explorados

```

8.3. Código de DFS

```

1 def depth_first_search(start, goal, get_neighbors):
2
3     fronterita = []
4     fronterita.append((start, [start]))
5     visitados = set()
6     visitados.add(start)
7     nodos_explorados = []
8     while fronterita:
9         pos, camin = fronterita.pop()
10        nodos_explorados.append(pos)
11        if pos == goal:
12            return camin, nodos_explorados
13        for vecino, costo in get_neighbors(pos):
14            if vecino not in visitados:
15                visitados.add(vecino)
16                if vecino == goal:
17                    return camin + [vecino], nodos_explorados
18                fronterita.append((vecino, camin + [vecino]))
19    return [], nodos_explorados
20

```

8.4. Código de UCS

```

1 def uniform_cost_search(start, goal, get_neighbors):
2     frontera = [(0, 0, start, [start])]
3     visitados = set()
4     contador = 0
5     nodos_explorados = []
6     while frontera:
7         costo, _, pos, camino = heapq.heappop(frontera)
8         if pos in visitados:
9             continue
10        visitados.add(pos)
11        nodos_explorados.append(pos)
12        if pos == goal:
13            return camino, nodos_explorados
14        for vecinos, costico_actual in get_neighbors(pos):
15            if vecinos not in visitados:
16                contador += 1
17                heapq.heappush(
18                    frontera,
19                    (costo + costico_actual, contador, vecinos, camino +
20                     [vecinos]),
21                )
22    return [], nodos_explorados

```

8.5. Código de bidireccional

```

1 def bidirectional_search(start, goal, get_neighbors):
2     fronterainicio = deque()
3     fronterainicio.append((start, [start]))
4     fronteragoal = deque()
5     fronteragoal.append((goal, [goal]))
6     visitadosinicio = {start: [start]}

```

```
7     visitadosgoal = {goal: [goal]}
8     exploradosi = []
9     exploradosg = []
10
11     while fronterainicio and fronteragoal:
12         i, camino_i = fronterainicio.popleft()
13         exploradosi.append(i)
14         if i in visitadosgoal:
15             return camino_i + visitadosgoal[i][::-1][1:], {
16                 "start": exploradosi,
17                 "goal": exploradosg,
18             }
19         for vecino, cos in get_neighbors(i):
20             if vecino not in visitadosinicio:
21                 visitadosinicio[vecino] = camino_i + [vecino]
22                 fronterainicio.append((vecino, camino_i + [vecino]))
23         j, camino_j = fronteragoal.popleft()
24         exploradosg.append(j)
25         if j in visitadosinicio:
26             return camino_j + visitadosinicio[j][::-1][1:], {
27                 "start": exploradosi,
28                 "goal": exploradosg,
29             }
30         for veci, costo in get_neighbors(j):
31             if veci not in visitadosgoal:
32                 visitadosgoal[veci] = camino_j + [veci]
33                 fronteragoal.append((veci, camino_j + [veci]))
34     return [], {"start": exploradosi, "goal": exploradosg}
```